

Python Veri Yapıları

Mehmet Hakan Satman (PhD, Professor of Numerical Methods)

March 20, 2020

İçindekiler

1.1	list	2
1.1.1	count	2
1.1.2	reverse	2
1.1.3	clear	3
1.1.4	sort	3
1.1.5	index	3
1.1.6	append	3
1.1.7	pop	4
1.1.8	insert	4
1.1.9	extend	4
1.1.10	remove	4
1.1.11	copy	5
1.1.12	İndis ile erişim	5
1.1.13	del	5
1.1.14	list comprehension	6
1.1.15	Operatörler	6
1.2	tuple	7
1.3	set	8
1.3.1	add	8
1.3.2	clear	9
1.3.3	pop	9
1.3.4	remove	9
1.3.5	union	10
1.3.6	intersection	10
1.3.7	difference	10
1.3.8	discard	10
1.4	dict	11
1.4.1	keys	11
1.4.2	values	11
1.4.3	get	11
1.4.4	clear, copy, pop	11
1.5	Veri nesnelerinin döngüler ile kullanımı	12
1.5.1	Liste ve döngüler	12
1.5.2	dict ve döngüler	14

1.1 list

`list` veri yapısı farklı tipte olabilen Python değişken veya değerlerinin saklanabildiği dinamik bir veri yapısıdır. Aşağıdaki örnekte bir Python listesi oluşturulmaktadır:

```
>>> a = [1, 5, 7, -1]
```

`list` tipindeki bir nesnenin ulaşılacak fonksiyon ve özellikleri `dir` fonksiyonu ile elde edilebilir:

```
>>> dir(a)
['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
```

Şimdi bu üye fonksiyonların bazılarına değinelim:

1.1.1 count

Bir liste üzerinden `count` fonksiyonu ile o listenin belirtilen elemandan kaç adet olduğu bilgisi elde edilir. Aşağıdaki örneği inceleyelim:

```
>>> a = [1, 2, 3, 3, 3, 3, 4, 4]
>>> a.count(1)
1
>>> a.count(2)
1
>>> a.count(3)
4
>>> a.count(4)
2
```

1.1.2 reverse

Bir listenin son elemanı ilk elemanı olacak şekilde tersini elde etmek için kullanılır. Düzenlemeler orijinal liste üzerinde gerçekleştirilir. Böylece işlem sonrasında orijinal sıralama kaydedilir. Aşağıdaki örneği inceleyelim:

```
>>> a = [1, 2, 3, 3, 3, 3, 4, 4]
>>> a.reverse()
>>> a
[4, 4, 3, 3, 3, 3, 2, 1]
```

1.1.3 clear

Bir Python listesinin içeriğinin silinmesi için kullanılır:

```
>>> a = [1, 2, 3, 3, 3, 3, 4, 4]
>>> a.clear()
>>> a
[]
```

Örnekten de anlaşılacağı üzere boş liste için [] ifadesi kullanılır.

1.1.4 sort

Bir Python listesinin sıralanması için kullanılabilecek bir fonksiyondur:

```
>>> a = [9, 8, 4, 0, 0, 1, 1, 7, 8, 3]
>>> a.sort()
>>> a
[0, 0, 1, 1, 3, 4, 7, 8, 8, 9]
```

1.1.5 index

Belirtilen elemanın listenin hangi elemanı olduğu bilgisini döndürür:

```
>>> a = [9, 8, 4, 0, 0, 1, 1, 7, 8, 3]
>>> a.index(3)
9
```

Burada, 3'ün indisi 9 olarak döndürülmüştür. Python'da liste indislerinin sıfırdan başladığı unutulmamalıdır.

1.1.6 append

Listenin sonuna eklemek için kullanılır:

```
>>> a = [9, 8, 4, 0, 0, 1, 1, 7, 8, 3]
>>> a.append(1000)
>>> a
[9, 8, 4, 0, 0, 1, 1, 7, 8, 3, 1000]
```

1.1.7 pop

Listenin son elemanını döndüren fonksiyondur. Ancak bu elemanı döndürmeden hemen önce bu elemanı listeden çıkartır:

```
>>> a = [9, 8, 4, 0, 0, 1, 1, 7, 8, 3]
>>> lastelement = a.pop()
>>>
>>> a
[9, 8, 4, 0, 0, 1, 1, 7, 8]
>>> lastelement
3
```

1.1.8 insert

Bir listenin belirtilen indisine bir elemanın yerleştirilmesini sağlar. Belirtilen indisteki eski elemanın ve sonrakilerin indisleri birer birim ötelenir.

```
>>> a.insert(0, 1000)
>>> a
[1000, 9, 8, 4, 0, 0, 1, 1, 7, 8, 3]
>>> a.insert(5, 42)
>>> a
[1000, 9, 8, 4, 0, 42, 0, 1, 1, 7, 8, 3]
```

1.1.9 extend

Bir listenin sonuna, başka bir listenin elemanlarının eklenmesi sağlanır:

```
>>> a = [9, 8, 4, 0, 0, 1, 1, 7, 8, 3]
>>> a.extend([-9, -8, -7])
>>> a
[9, 8, 4, 0, 0, 1, 1, 7, 8, 3, -9, -8, -7]
```

1.1.10 remove

Bir listenin belirtilen bir elemanını listeden çıkartır. Burada elemanın indisi değil kendi değeri argüman olarak verilir. Bu elemandan birden fazla varsa ilki silinir:

```
>>> a
[9, 8, 4, 0, 0, 1, 1, 7, 8, 3, -9, -8, -7]
>>> a.remove(-9)
>>> a.remove(0)
>>> a.remove(1)
>>> a
[9, 8, 4, 0, 1, 7, 8, 3, -8, -7]
```

Silinmek istenen eleman listede yoksa bir hata alınır:

```
>>> a
[9, 8, 4, 0, 1, 7, 8, 3, -8, -7]
>>> a.remove(100)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

Bu yüzden öncelikle listenin belirtilen elemana sahip olup olmadığı kontrol edilebilir:

```
>>> if a.count(42) > 0:
...     a.remove(42)
```

1.1.11 copy

Bir listenin kopyasını döndüren üye fonksiyondur:

```
>>> a
[9, 8, 4, 0, 1, 7, 8, 3, -8, -7]
>>> b = a.copy()
>>> b
[9, 8, 4, 0, 1, 7, 8, 3, -8, -7]
```

1.1.12 İndis ile erişim

Listelerin elemanlarına indis operatörü ile erişilebilir:

```
>>> a
[9, 8, 0, 1, 7, 8, 3, -8, -7]
>>> a[0]
9
>>> a[1]
8
>>> a[2]
0
```

Ayrıca listenin belirtilen aralıktaki elemanlarının bir listesine de erişilebilir. Burada : operatörü bir aralık belirtir. Aralığın ilk elemanı dahildir ancak son elemanı dahil değildir. 0:3 gibi bir aralık listenin 0., 1. ve 2. elemanlarının oluşturduğu bir liste döndürür.

```
>>> a
[9, 8, 0, 1, 7, 8, 3, -8, -7]
>>> a[0:3]
[9, 8, 0]
```

1.1.13 del

Listeye ait olmayan bu direktif ile bir Python listesinin belirtilen elemanı listeden silinebilir:

```
>>> a
[9, 8, 4, 0, 1, 7, 8, 3, -8, -7]
>>> del a[2]
>>> a
[9, 8, 0, 1, 7, 8, 3, -8, -7]
```

1.1.14 list comprehension

Python listeleri belirli bir örüntü oluşturacak şekilde tanımlanabilir. Aşağıdaki örneği inceleyelim:

```
>>> a = [i**2 for i in range(1, 10)]
>>> a
[1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Örnekte i 'nin 1, 2, ..., 9 değerleri için i^2 'nin bir listesi oluşturulur. Aşağıdaki örneği inceleyelim:

```
>>> a = [i ** 2 for i in range(1, 10) if i % 2 == 0]
>>> a
[4, 16, 36, 64]
```

Bu kez de i 'nin 1, 2, ..., 9 değerleri içinden çift olanlarının bir listesi oluşturulmuştur. Aşağıdaki örnekte $i = 1, 2, 3, 4, 5$ ve $j = 1, 2, 3, 4, 5$ için çarpım tablosu değerleri elde edilmektedir:

```
>>> a = [(i, j, i * j) for i in range(1, 6) for j in range(1, 6)]
>>> a
[(1, 1, 1), (1, 2, 2), (1, 3, 3), (1, 4, 4), (1, 5, 5), (2, 1, 2), (2, 2, 4),
(2, 3, 6), (2, 4, 8), (2, 5, 10), (3, 1, 3), (3, 2, 6), (3, 3, 9), (3, 4, 12),
(3, 5, 15), (4, 1, 4), (4, 2, 8), (4, 3, 12), (4, 4, 16), (4, 5, 20),
(5, 1, 5), (5, 2, 10), (5, 3, 15), (5, 4, 20), (5, 5, 25)]
```

(x, y, z) gibi parantez içinde gösterilen bu ifadeler `tuple` tipindedir ve sonraki bölümde incelenecektir.

1.1.15 Operatörler

Python listeleri için bir takım operatörler tanımlanmıştır. Şimdi bunlardan birkaçını inceleyelim. `+` operatörü ile iki listenin birleştirilmesi sağlanır. Sonuç olarak yeni bir liste döndürülür. Var olan listelerin içerikleri bozulmaz:

```
>>> a = [1, 2, 3]
>>> b = [9, 8, 7]
>>> c = a + b
>>> c
[1, 2, 3, 9, 8, 7]
```

`*` operatörü ile bir listenin n kez kendisiyle birleştirilmesi sağlanır:

```
>>> a = [1,2,3]
>>> b = a * 5
>>> b
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
```

`in` operatörü ile bir elemanın bir liste içinde olup olmadığı test edilir. Eleman listede varsa `True` aksi durumda `False` döndürülür:

```
>>> a
[1, 2, 3]
>>> 2 in a
True
>>> 6 in a
False
>>> 10 in [1,2,3,4,5,6,7,8,9,10]
True
```

1.2 tuple

Parantez ile $(x, y, \dots, z)$ şeklinde tanımlanan ifadeler Python'da `tuple` tipindedir:

```
>>> a = (1,2,3)
>>> type(a)
<class 'tuple'>
```

`count` ve `index` fonksiyonları bir `tuple` nesnesi üzerinden listelerde olduğu gibi kullanılabilir:

```
>>> a = (1,2,3)
>>> a.count(1)
1
>>> a.index(3)
2
```

`tuple` tipindeki veri nesneleri listeler gibi davranır ancak değerleri değiştirilemez, yeni veri eklenemez ve veri silinemez (immutable). Program akışı esnasında içeriğinin değişmemesi gereken veriler için güvenlik gibi sebeplerle tercih edilebilir. Bunun yanında bir `tuple` verisinin elemanlarına listelerde olduğu gibi kolaylıkla erişilir:

```
>>> a = (1, -2, 9)
>>> a[0]
1
>>> a[1]
-2
>>> a[2]
9
>>> len(a)
3
```

`tuple` tipindeki veri nesnelerinin elemanlarının değiştirilmesi istenirse hata alınır:

```
>>> a = (1, -2, 9)
>>> a[0] = 100
```



```
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

Bir tuple nesnesi tek bir eleman içerecek ise bu elemandan sonra virgül kullanılmalıdır. Aksi durumda bir tuple yerine içeriğin kendisi elde edilir:

```
>>> a = (30)
>>> a
30
>>> a = (30, )
>>> a
(30,)
```

list fonksiyonu ile bir tuple nesnesinin elemanlarından yeni bir liste oluşturulur:

```
>>> a = (42, 39, 80)
>>> b = list(a)
>>> b
[42, 39, 80]
>>> type(b)
<class 'list'>
```

Bir listeyi de tuple fonksiyonu ile tuple tipinde bir nesne elde etmek için kullanabiliriz:

```
>>> mylist = [6, 8, 10, 12]
>>> t = tuple(mylist)
>>> t
(6, 8, 10, 12)
>>> type(t)
<class 'tuple'>
```

1.3 set

set veri yapısı her elemanın yalnızca bir kez bulunmasının izin verildiği bir veri yapısıdır. Küme parantezi ile tanımlanabilir:

```
>>> myset = {"ekonometri", "istatistik", "yoneylem arastirmasi"}
>>> type(myset)
<class 'set'>
>>> myset
{'istatistik', 'yoneylem arastirmasi', 'ekonometri'}
```

set veri yapısında veriler, kaydedildikleri sırada tutulmak zorunda değildir. Bu halleriyle, amaca uygun olarak, matematikteki küme kavramına yakındır.

1.3.1 add

Bir set nesnesine yeni bir eleman eklemek için kullanılır. Var olan eleman öncesinde varsa ekleme yapılmaz:

```
>>> myset = {"ekonometri", "istatistik", "yoneylem"}
>>> myset.add("iktisat")
>>> myset.add("isletme")
>>> myset
{'yoneylem', 'iktisat', 'istatistik', 'ekonometri', 'isletme'}
>>> myset.add("isletme")
>>> myset
{'yoneylem', 'iktisat', 'istatistik', 'ekonometri', 'isletme'}
```

1.3.2 clear

Listelerde olduğu gibi içeriğin temizlenmesi için kullanılır:

```
>>> myset = {"ekonometri", "istatistik", "yoneylem"}
>>> myset.clear()
>>> myset
set()
```

Görüldüğü gibi `set()` ifadesi boş bir `set` tanımlar.

1.3.3 pop

Listelerdekine benzer şekilde çalışır. Ancak `set` nesneleri sıralı olmadığı için döndürülecek elemanın tesadüfi olduğu düşünülebilir:

```
>>> myset = {"ekonometri", "istatistik", "yoneylem"}
>>> myset.pop()
'istatistik'
>>> myset.pop()
'yoneylem'
>>> myset.pop()
'ekonometri'
>>> myset.pop()
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
KeyError: 'pop from an empty set'
```

`set` içinde herhangi bir eleman kalmadığında, görüldüğü gibi bir hata alınır.

1.3.4 remove

Belirtilen bir elemanın `set` içinden silinmesi için kullanılır, belirtilen eleman yoksa bir hata alınır:

```
>>> myset = {"ekonometri", "istatistik", "yoneylem"}
>>> myset.remove("yoneylem")
>>> myset
{'istatistik', 'ekonometri'}
>>> myset.remove("iktisat")
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
```

```
KeyError: 'iktisat'
```

1.3.5 union

İki set nesnesinin birleşim kümesini bir set nesnesi olarak döndürür:

```
>>> myset = {"ekonometri", "istatistik", "yoneylem"}
>>> myotherset = {"iktisat", "isletme", "yoneylem"}
>>>
>>> birlesim = myset.union(myotherset)
>>> birlesim
{'yoneylem', 'istatistik', 'ekonometri', 'iktisat', 'isletme'}
```

1.3.6 intersection

set nesnesinin belirtilen başka bir set nesnesiyle olan kesişim kümesi yine bir set olarak döndürülür:

```
>>> myset = {"ekonometri", "istatistik", "yoneylem"}
>>> myotherset = {"iktisat", "isletme", "yoneylem"}
>>> myset.intersection(myotherset)
{'yoneylem'}
```

1.3.7 difference

set nesnesinin belirtilen bir set nesnesinden farkının kümesini yine bir set olarak döndürür:

```
>>> myset = {"ekonometri", "istatistik", "yoneylem"}
>>> myotherset = {"iktisat", "isletme", "yoneylem"}
>>> myset.difference(myotherset)
{'istatistik', 'ekonometri'}
```

1.3.8 discard

Belirtilen bir elemanın set nesnesinden silinmesi sağlanır. Eleman yoksa hata alınmaz.

```
>>> myset = {"ekonometri", "istatistik", "yoneylem"}
>>> myset.discard("yoneylem")
>>> myset
{'istatistik', 'ekonometri'}
>>> myset.discard("yoneylem")
>>> myset
{'istatistik', 'ekonometri'}
```

1.4 dict

dict veri yapısı **anahtar: deger** şeklinde veriler tutmak için kullanılır. Listelerden farklı olarak indis olarak belirtilen anlamlı anahtar değerler kullanılabilir:

```
>>> sozluk = {"go": "git", "come": "gel"}
>>> sozluk["go"]
'git'
>>> sozluk["come"]
'gel'
>>> type(sozluk)
<class 'dict'>
```

Alt bölümlerde dict veri yapısının sahip olduğu bazı üye fonksiyonlara değineceğiz.

1.4.1 keys

dics nesnesindeki anahtar değerleri döndürür:

```
>>> notlar.keys()
dict_keys(['matematik', 'ekonometri', 'yoneylem'])
>>> list(notlar.keys())
['matematik', 'ekonometri', 'yoneylem']
```

1.4.2 values

dics nesnesindeki değerleri döndürür:

```
>>> notlar = {"matematik": 80, "ekonometri": 90, "yoneylem": 75}
>>> notlar.values()
dict_values([80, 90, 75])
>>> list(notlar.values())
[80, 90, 75]
```

1.4.3 get

İndis operatörü gibi, belirli bir anahtar değere karşılık gelen değeri döndürür:

```
>>> notlar = {"matematik": 80, "ekonometri": 90, "yoneylem": 75}
>>> notlar.get("matematik")
80
>>> notlar["matematik"]
80
```

1.4.4 clear, copy, pop

Sırasıyla içeriği temizlemek, içeriğin bir kopyasını elde etmek ve belirli bir elemanı döndürüp listeden temizlemek için kullanılırlar:

```

>>> notlar = {"matematik": 80, "ekonometri": 90, "yoneylem": 75}
>>> ekonot = notlar.pop("ekonometri")
>>> ekonot
90
>>> notlar
{'matematik': 80, 'yoneylem': 75}
>>> notlar.clear()
>>> notlar
{}

```

1.5 Veri nesnelerinin döngüler ile kullanımı

Python'da veri yapıları ve döngüler arasında sıkı bir ilişki vardır. Tek bir eleman için tanımlı olan işlemler veri yapılarının tüm elemanlarına döngüler yoluyla uygulanabilir. not: bir diğer yaklaşım olarak fonksiyonel programlama ve özyineleme (recursion) incelenebilir.

1.5.1 Liste ve döngüler

Listelerin elemanları için tanımlı bir işlem `for` döngüleri ile listenin tüm elemanlarına uygulanabilir:

```

>>> a = [1,2,42, 39, 80]
>>> for i in a:
...     print(i)
...
1
2
42
39
80

```

Aşağıdaki program ile bir listenin elemanlarının karelerinin bir listesi oluşturulmaktadır:

```

>>> a = [1,2,42, 39, 80]
>>> b = []
>>> for i in a:
...     b.append(i ** 2)
...
>>> b
[1, 4, 1764, 1521, 6400]

```

Aynı işlemi aşağıdaki gibi gerçekleştirebilirdik:

```

>>> a = [1,2,42, 39, 80]
>>> b = [i ** 2 for i in a]
>>>
>>> a
[1, 2, 42, 39, 80]
>>> b
[1, 4, 1764, 1521, 6400]

```

b listesini oluşturunmanın diğer bir yolu da `map` fonksiyonunu kullanmaktır:

```
>>> def karele(x):
...     return x ** 2
...
>>> b = map(karele, a)
>>> b
<map object at 0x104e0da90>
```

`map` fonksiyonundan dönen değerin tipi `map_object` olmaktadır ancak bu değer bir liste olarak da elde edilebilir:

```
>>> def karele(x):
...     return x ** 2
...
>>> b = list(map(karele, a))
>>> b
[1, 4, 1764, 1521, 6400]
```

Karele fonksiyonu bir `lambda` ifade olarak tanımlanıp `map` içinde doğrudan kullanılabilir:

```
>>> a = [1,2,42, 39, 80]
>>> b = list(map(lambda x: x ** 2, a))
>>> b
[1, 4, 1764, 1521, 6400]
```

`filter` fonksiyonu ile bir listenin elemanlarından belirli bir koşulu sağlayanlarının bir listesi oluşturulabilir. Örneğin bir listenin çift elemanlarını filtreleyen bir Python kodunu aşağıdaki şekilde yazabiliriz:

```
>>> def ciftmi(x):
...     return x % 2 == 0
...
>>> a = [1,2,42, 39, 80]
>>> b = filter(ciftmi, a)
>>> b
<filter object at 0x104e0db10>
>>> list(b)
[2, 42, 80]
```

Sonuçlar doğrudan bir liste olarak alınabilir:

```
>>> def ciftmi(x):
...     return x % 2 == 0
...
>>> a = [1,2,42, 39, 80]
>>> b = list(filter(ciftmi, a))
>>> b
[2, 42, 80]
```

`ciftmi` fonksiyonunu kullanmak yerine bir `lambda` ifadesi `filter` fonksiyonu içinde doğrudan kullanılabilir:

```
>>> a = [1,2,42, 39, 80]
>>> b = list(filter(lambda x: x % 2 == 0, a))
>>> b
```

[2, 42, 80]

1.5.2 dict ve döngüler

dict tipindeki veri nesneleri anahtar ve değer şeklinde veri tuttuğundan döngülerle birlikte farklı kullanımları söz konusudur. Aşağıdaki dict nesnesini ele alalım:

```
>>> d = {"tr": "Turkce", "en": "English", "fr": "Franch", "g": "German"}
```

Bu veri yapısı üzerinden bir for döngüsü aşağıdaki şekilde kurulabilir:

```
>>> for eleman in d:  
...     print(eleman)  
...  
tr  
en  
fr  
g
```

Görüldüğü gibi for döngüsü ile anahtar değerlerine ulaşılabilir. Anahtar değerler alındığına göre bu anahtarlar karşılık gelen değerlere aşağıdaki gibi ulaşılabilir:

```
>>> for eleman in d:  
...     print(d[eleman])  
...  
Turkce  
English  
Franch  
German
```

map fonksiyonu ile tüm anahtar değerler için bir işlemin uygulanması sağlanabilir:

```
>>> def f(x):  
...     return x + "!"  
...  
>>> list(map(f, d))  
['tr!', 'en!', 'fr!', 'g!']
```

Benzer işlemler anahtarlar yerine değerlere de uygulanabilir:

```
>>> def f(x):  
...     return d[x]  
...  
>>> list(map(f, d))  
['Turkce', 'English', 'Franch', 'German']
```